data-pipelining-with-polygon

October 29, 2026

0.1 Post Updates

Update 8/6/2026: Brought the post back in line with the current source. Polygon.io has rebranded to Massive and the Python client is now imported as `massive`, `polygon_pull_data` gained a `parquet_export` parameter and returns the resulting DataFrame, and `polygon_fetch_full_history` renamed its `full_history_df` parameter to `existing_history_df`. Replaced the reproduced docstrings with prose descriptions, added the free tier constraints that shape the design, and documented the on-disk layout the data lands in.

0.2 Introduction

Where the previous post on collecting crypto asset data from Coinbase covered digital assets, this one covers the equity side. The two functions below pull minute, hour, and daily bars for equities and ETFs from Polygon.io, since rebranded as Massive.

Two constraints from the free tier drive most of the design. The first is a two year limit on historical data, which is why the examples below start from a date two years back rather than attempting to pull an asset's full history. The second is a rate limit of five API requests per minute, which is what the `free_tier` flag handles. When using the free tier, the fetch pauses for twelve seconds between requests. A single API call also returns at most 5,000 records, so any meaningful date range has to be walked through in chunks and reassembled.

The work is split across two functions that build on one another. `polygon_fetch_full_history` walks a date range and returns it as a single dataframe. `polygon_pull_data` wraps that to maintain pickle, Excel, and Parquet files on disk, extending an existing record with the rows it is missing, or creating a new record where none exists.

0.3 Python Imports

[1]:
```
# Standard Library
import os
import sys
import time
import warnings

from datetime import datetime
from pathlib import Path

# Data Handling
import pandas as pd
```

```
# Suppress warnings
warnings.filterwarnings("ignore")
```

[2]:
```
# Add the source subdirectory to the system path to allow import config from␣
 ↪settings.py
current_directory = Path(os.getcwd())
website_base_directory = current_directory.parent.parent.parent
src_directory = website_base_directory / "src"
sys.path.append(str(src_directory)) if str(src_directory) not in sys.path else␣
 ↪None

# Import settings.py
from settings import config

# Add configured directories from config to path
SOURCE_DIR = config("SOURCE_DIR")
sys.path.append(str(Path(SOURCE_DIR))) if str(Path(SOURCE_DIR)) not in sys.path␣
 ↪else None

# Add other configured directories
BASE_DIR = config("BASE_DIR")
CONTENT_DIR = config("CONTENT_DIR")
POSTS_DIR = config("POSTS_DIR")
PAGES_DIR = config("PAGES_DIR")
PUBLIC_DIR = config("PUBLIC_DIR")
SOURCE_DIR = config("SOURCE_DIR")
DATA_DIR = config("DATA_DIR")
DATA_MANUAL_DIR = config("DATA_MANUAL_DIR")
```

0.4 Python Functions

Here are the functions needed for this project, listed in the order they build on one another:

- polygon_fetch_full_history: Fetch full historical data for a given product from Polygon API.
- polygon_pull_data: Read existing data file, download price data from Polygon, and export data.

[3]:
```
from polygon_fetch_full_history import polygon_fetch_full_history
from polygon_pull_data import polygon_pull_data
```

One naming note before the code: Polygon.io has rebranded to Massive, and the official Python client followed. The package is now `massive` rather than `polygon-api-client`, so the import below reads `from massive import RESTClient`. The API surface is otherwise unchanged, and I've kept "Polygon" in the function names and directory layout since that's what the existing data is filed under.

0.5 Function Usage

Both examples below use daily bars, but switching to minute or hour data is a matter of changing the `timespan` and `multiplier` arguments, with everything else unchanged. Note that finer granularity burns through the free tier rate limit considerably faster, since each request covers a much shorter window. Massive actually aggregates minute data into hour bars, so the time to pull minute and hour data is actually the same. Both functions accept only `minute`, `hour`, or `day` for `timespan` and raise an exception otherwise, since the window sizing logic is only defined for those three.

0.5.1 Polygon Fetch Full History

This is the lower level of the two functions. Starting from `current_start`, it walks forward through the date range in fixed size windows - 180 days for daily data, 15 days for minute and hour data - requests each window from the API, and appends the results to the dataframe passed in as `existing_history_df`, stopping once it reaches the present. Because each request is capped at 5,000 records, ranges spanning years are handled the same way as the single window below: the individual responses are concatenated and returned as one dataframe.

The API key is read through `config`, which merges everything in the `.env` file into the settings defaults, so `config("POLYGON_KEY")` resolves without any further setup:

[4]:
```
from massive import RESTClient

# Load API key from the environment variables
POLYGON_KEY = config("POLYGON_KEY")

# Get the environment variable for where data is stored
DATA_DIR = config("DATA_DIR")

# Open client connection
client = RESTClient(api_key=POLYGON_KEY)

# Create an empty DataFrame
df = pd.DataFrame({
    'Date': pd.Series(dtype="datetime64[ns]"),
    'open': pd.Series(dtype="float64"),
    'high': pd.Series(dtype="float64"),
    'low': pd.Series(dtype="float64"),
    'close': pd.Series(dtype="float64"),
    'volume': pd.Series(dtype="float64"),
    'vwap': pd.Series(dtype="float64"),
    'transactions': pd.Series(dtype="int64"),
    'otc': pd.Series(dtype="object")
})

# Example usage - daily
df = polygon_fetch_full_history(
    client=client,
    ticker="AMZN",
```

```
    timespan="day",
    multiplier=1,
    adjusted=True,
    existing_history_df=df,
    current_start=datetime(2025, 1, 1),
    free_tier=True,
    verbose=False,
)

time.sleep(12)  # Sleep for 12 seconds to avoid hitting rate limits
```

The example above pulls daily data starting from 1/1/2025. Here are the first five rows:

[5]:
```
display(df.head())
```

```
                 Date     open     high     low   close      volume      vwap  \
0 2025-01-02 05:00:00  222.030  225.150  218.19  220.22  33956579.0  221.2745
1 2025-01-03 05:00:00  222.505  225.360  221.62  224.19  27503606.0  223.7046
2 2025-01-06 05:00:00  226.780  228.835  224.84  227.61  31849831.0  227.0921
3 2025-01-07 05:00:00  227.900  228.381  221.46  222.11  28084164.0  223.4033
4 2025-01-08 05:00:00  223.185  223.520  220.20  222.13  25033292.0  222.0414

   transactions   otc
0        449631  None
1        346975  None
2        410686  None
3        379570  None
4        325539  None
```

0.5.2 Polygon Pull Data

This function wraps the one above and performs the following:

1. Attempt to read an existing pickle data file.
2. If a data file exists, then pull updated data starting from the last date on record.
3. Otherwise, pull all historical data available for that asset for the past 2 years (or chosen interval).
4. Store Pickle, Excel, and/or Parquet files of the data in the specified directories.
5. Return the resulting dataframe.

Setting `force_existing_check` re-pulls the entire range rather than starting from the last date on record, which is how gaps in an existing file get filled.

The `base_directory`, `source`, `asset_class`, and `timespan` arguments together tell the function where in the local filesystem to look for an existing pickle file, and where to store the updated exports. They assemble into a path of the form `{base_directory}/{source}/{asset_class}/{timespan}/{ticker}`, so the example below reads and writes:

```
Data/Polygon/Equities/day/AMZN.pkl
Data/Polygon/Equities/day/AMZN.xlsx
```

Data/Polygon/Equities/day/AMZN.parquet

Keeping the timespan in the path means minute, hour, and daily records for the same ticker sit side by side without overwriting one another.

[6]:
```
current_year = datetime.now().year
current_month = datetime.now().month
current_day = datetime.now().day

# Example usage - daily
df = polygon_pull_data(
    base_directory=DATA_DIR,
    ticker="AMZN",
    source="Polygon",
    asset_class="Equities",
    start_date=datetime(current_year - 2, current_month, current_day),
    timespan="day",
    multiplier=1,
    adjusted=True,
    force_existing_check=True,
    free_tier=True,
    verbose=False,
    excel_export=True,
    pickle_export=True,
    parquet_export=True,
    output_confirmation=True,
)

time.sleep(12)  # Sleep for 12 seconds to avoid hitting rate limits
```

```
File found…updating the AMZN day data.
Last date in existing data: 2026-10-29 04:00:00
Number of rows in existing data: 818
Forcing check of existing data…

Exporting AMZN day data to Excel…
Exporting AMZN day data to Pickle…
Exporting AMZN day data to Parquet…

The first and last date of day data for AMZN is:

                 Date    open    high     low   close      volume      vwap  \
0 2023-07-28 04:00:00  129.69  133.01  129.33  132.21  46269781.0  131.8837

   transactions   otc
0        413438  None

                   Date    open    high     low   close        volume  \
817 2026-10-29 04:00:00  246.95  249.14  241.24  246.36  2.899970e+07
```

```
         vwap  transactions   otc
817  246.0853        458904  None

Number of rows after data update: 818
Number of rows added during update: 0
Polygon data complete for AMZN day data.
--------------------
```

And here are the first five rows of the returned dataframe:

[7]:
```
display(df.head())
```

```
                  Date     open    high       low   close      volume  \
0 2023-07-28 04:00:00  129.690  133.01  129.3300  132.21  46269781.0
1 2023-07-31 04:00:00  133.200  133.87  132.3800  133.68  41901516.0
2 2023-08-01 04:00:00  133.550  133.69  131.6199  131.69  42250989.0
3 2023-08-02 04:00:00  130.154  130.23  126.8200  128.21  50988614.0
4 2023-08-03 04:00:00  127.480  129.84  126.4100  128.91  90855736.0

       vwap  transactions   otc
0  131.8837        413438  None
1  133.3410        406644  None
2  132.2470        385743  None
3  128.3973        532942  None
4  131.4941        746639  None
```

Note that `Date` is returned as a regular column with a plain integer index rather than as the index itself. That is deliberate, and any downstream analysis can call `set_index("Date")` as needed.

0.6 Future Investigation

1. Calculate adjusted open/high/low/close prices using the dividend adjustment factor.

0.7 References

1. Massive (formerly Polygon.io)
2. Massive REST API quickstart
3. massive on PyPI
4. massive-com/client-python on GitHub
5. Reusable & Extensible Python Functions For Financial Data Analysis